



PUC Minas

Pontifícia Católica de Minas Gerais Coração Eucarístico

Análise de Eficácia de Testes com Teste de Mutação

Professor: Cleiton Silva Tavares

Aluna: Luisa Clara de Paula Lara Silva

Matrícula: 814542

Análise Inicial

No primeiro momento após instalar as devidas dependências e rodar o npm test existiam 50 testes e 1 suíte de teste total, na análise igual demonstrado no print seguinte tínhamos 85.41% de stms, 98.64% de cobertura de linha e 100% de função com apenas uma linha sem cobertura a 112

```
✓ 49. deve calcular o triplo de um número
✓ 50. deve calcular a metade de um número

-----|-----|-----|-----|-----|-----
File    | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files | 85.41   | 58.82    | 100     | 98.64   |
operacoes.js | 85.41   | 58.82    | 100     | 98.64   | 112
-----|-----|-----|-----|-----|-----

Test Suites: 1 passed, 1 total
Tests:       50 passed, 50 total
Snapshots:   0 total
Time:        0.552 s, estimated 1 s
Ran all test suites.
PS C:\Users\luisa\OneDrive\Área de Trabalho\facul ou trabalho\twstesoft\mutantes\operacoes-mutante> |
```

Após instalar o stryker e roda-lo criou-se 213 mutações, e igual demonstrado nos prints seguintes do relatório do stryker html, dos 213 mataram 154 sobreviveram 44, 3 timeouts e 12 eram mutantes que não tinha cobertura, gerando uma porcentagem de 78,11% de cobertura, ao comparar com a porcentagem de linha com cobertura temos uma diferença de 20%. Uma diferença muito grande e interessante é que esses 12% de linha sem cobertura foram o suficiente para criar 12 erros por não cobertura. Mostrando também que mesmo com 50 testes diferentes muitos mutantes ainda conseguiram sobreviver

All files

Mutants

Tests

All files

1574412

File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
All files	73.71	78.11	154	44	3	12	0	0	0	157	56	213
js operacoes.js	73.71	78.11	154	44	3	12	0	0	0	157	56	213

Análise de Mutantes Críticos e Solução Implementada

Mutante 1 linha 44:

```
43 function isPar(n) { return n % 2 === 0; }
44 - function isImpar(n) { return n % 2 !== 0; }
+ function isImpar(n) { return n * 2 !== 0; }
45 function calcularPorcentagem(percentual, valor) { return (valor * percentual) / 100; }
46 function aumentarPorcentagem(valor, percentual) { return valor * (1 + percentual / 100); }
47 function diminuirPorcentagem(valor, percentual) { return valor * (1 - percentual / 100); }
48 function inverterSinal(n) { return -n; }
49
50 // === Bloco 3: Funções Trigonômicas e Logarítmicas (21-30) ===
51 function seno(anguloRad) { return Math.sin(anguloRad); }
52 function cosseno(anguloRad) { return Math.cos(anguloRad); }
53 function tangente(anguloRad) { return Math.tan(anguloRad); }
54 function logaritmoNatural(n) { return Math.log(n); }
55 function logaritmoBase10(n) { return Math.log10(n); }
56 function arredondarParaBaixo(n) { return Math.floor(n); }
57 function arredondarParaCima(n) { return Math.ceil(n); }

ArithmeticOperator Survived (44:30) More
Covered by 1 test (yet still survived)
```

O teste só verificava o resultado true com o valor 7. Com o mutante: $7 * 2 = 14$, e $14 !== 0$ ainda é true — resultado idêntico ao original. O Stryker não conseguiu distinguir os dois, porque o teste nunca verificou um caso que retornasse false. Mudei para `expect(isImpar(4)).toBe(false)` para matar o mutante, pois $4 * 2 = 8 !== 0$ (resultado como mutante seria true)

Mutante 2 linha 74:

```
73 if (n <= 1) return false;
74 - for (let i = 2; i < n; i++) {
+ for (let i = 2; i >= n; i++) {
75     if (n % i === 0) return false;
76 }
77 return true;
78 }
79 function fibonacci(n) { // Retorna o n-ésimo termo
80     if (n <= 1) return n;
81     return fibonacci(n - 1) + fibonacci(n - 2);
82 }
83 function produtoArray(numeros) {
84     if (numeros.length === 0) return 1;
85     return numeros.reduce((acc, val) => acc * val, 1);
86 }
```

Com $i \geq n$ e $n = 7$, a condição $2 \geq 7$ é imediatamente false — o loop nunca executa, e a função retorna true direto. Isso é a resposta correta para 7, então o teste não detectou nada errado. O mutante matou completamente a lógica de verificação de primalidade, mas o teste escolheu justamente um número primo — nunca testou um composto como `isPrimo(4)`, que deveria ser false mas com o mutante retornaria true.

Mutante 3 linha 93:

```

77   return true;
78 }
79 function fibonacci(n) { // Retorna o n-ésimo termo
80   if (n <= 1) return n;
81   return fibonacci(n - 1) + fibonacci(n - 2);
82 }
83 function produtoArray(numeros) {
84   if (numeros.length === 0) return 1;
85   return numeros.reduce((acc, val) => acc * val, 1);
86 }
87 function clamp(valor, min, max) {
88   if (valor < min) return min;
89   if (valor > max) return max;
90   return valor;
91 }
92 function isDivisivel(dividendo, divisor) { return dividendo % divisor === 0; }
93 - function celsiusParaFahrenheit(celsius) { return (celsius * 9/5) + 32; }
94 + function celsiusParaFahrenheit(celsius) { return (celsius * 9 * 5) + 32; }
95 function fahrenheitParaCelsius(fahrenheit) { return (fahrenheit - 32) * 5/9; }
96 function inverso(n) {
97   if (n === 0) throw new Error('Não é possível inverter o número zero.');
```

O único valor testado foi 0°C. Qualquer fórmula multiplicada por zero dá zero: $0 * 9/5 + 32 = 32$ e $0 * 9 * 5 + 32 = 32$ — o resultado é idêntico. O mutante quebrou completamente a fórmula de conversão, mas o teste escolheu o único valor de entrada que torna qualquer multiplicação irrelevante. Com `celsiusParaFahrenheit(100)` o original daria 212 e o mutante daria 4532 — a diferença seria imediatamente detectada.

Resultados Finais

Após diversas modificações (foram diversas tentativas e criação de 32 novos casos de teste para bater a meta de 98% de mutantes mortos, após chegar em 96% notei que essa era a porcentagem máxima possível de chegar em modificar o código principal, nesse cenário foi obtido 100% de cobertura por linha. Como demonstrado nos prints seguintes:

```

> operacoes-mutante-lab@1.0.0 test
> jest --coverage
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	91.66	76.47	100	99.32	
.stryker-tmp/sandbox-CsxnFL/src	85.41	58.82	100	98.64	
operacoes.js	85.41	58.82	100	98.64	113
src	97.91	94.11	100	100	
operacoes.js	97.91	94.11	100	100	19,84

```

Test Suites: 2 passed, 2 total
Tests:       123 passed, 123 total
Snapshots:   0 total
Time:        0.634 s, estimated 1 s
```

```
Ran 2.28 tests per mutant on average.
```

File	% Mutation score		# killed	# timeout	# survived	# no cov	# errors
	total	covered					
All files	96.71	96.71	203	3	7	0	0
operacoes.js	96.71	96.71	203	3	7	0	0

Após modificar o código original foi possível alcançar 99% de mutantes mortos, isso ocorreu pois, dois if geraram 5 mutantes redundantes, mutações que alteram o código mas nunca mudam o resultado observável.

```
Tests ran:
```

```
Suíte de Testes Fraca para 50 Operações Aritméticas 36. deve manter um valor dentro de um intervalo (clamp)  
Suíte de Testes Fraca para 50 Operações Aritméticas 61. clamp retorna max quando valor é maior que o máximo
```

```
Ran 2.32 tests per mutant on average.
```

File	% Mutation score		# killed	# timeout	# survived	# no cov	# errors
	total	covered					
All files	99.01	99.01	197	4	2	0	0
operacoes.js	99.01	99.01	197	4	2	0	0

```
21:04:54 (4116) INFO HtmlReporter Your report can be found at: file:///C:/Users/luisa/OneDrive/%C3%81rea%20de%20Trabalho/facul%20ou%20trabalho/twstesoftware/mutantes/operacoes-mutante/reports/mutation/mutation.html
```

```
21:04:54 (4116) INFO MutationTestExecutor Done in 11 seconds.
```

```
PS C:\Users\luisa\OneDrive\Área de Trabalho\facul ou trabalho\twstesoftware\mutantes\operacoes-mutante>
```

Conclusão

Com esse trabalho deu pra ver que só olhar a porcentagem de cobertura de linha não garante que o teste tá bom de verdade. No começo, o code coverage era de 98,64%, mas o Stryker acusou que 20% dos mutantes ainda sobreviviam. Isso aconteceu porque muitos testes eram fracos. Isso prova que o teste de mutação é muito mais eficiente do que as métricas comuns que costumamos usar.

Além de ajudar a criar testes melhores, precisei criar 32 novos casos para chegar no resultado final. sendo ainda necessário modificar o código original, a fim de bater a meta de mais de 98% de mutantes mortos, devido serem mutantes redundantes, sendo possível no final conseguir 99,01% de mutantes mortos. No fim das contas, o teste de mutação não só deixa o sistema mais seguro, mas também força a gente a escrever um código mais limpo e direto ao ponto.